

One Definition, Every Pump

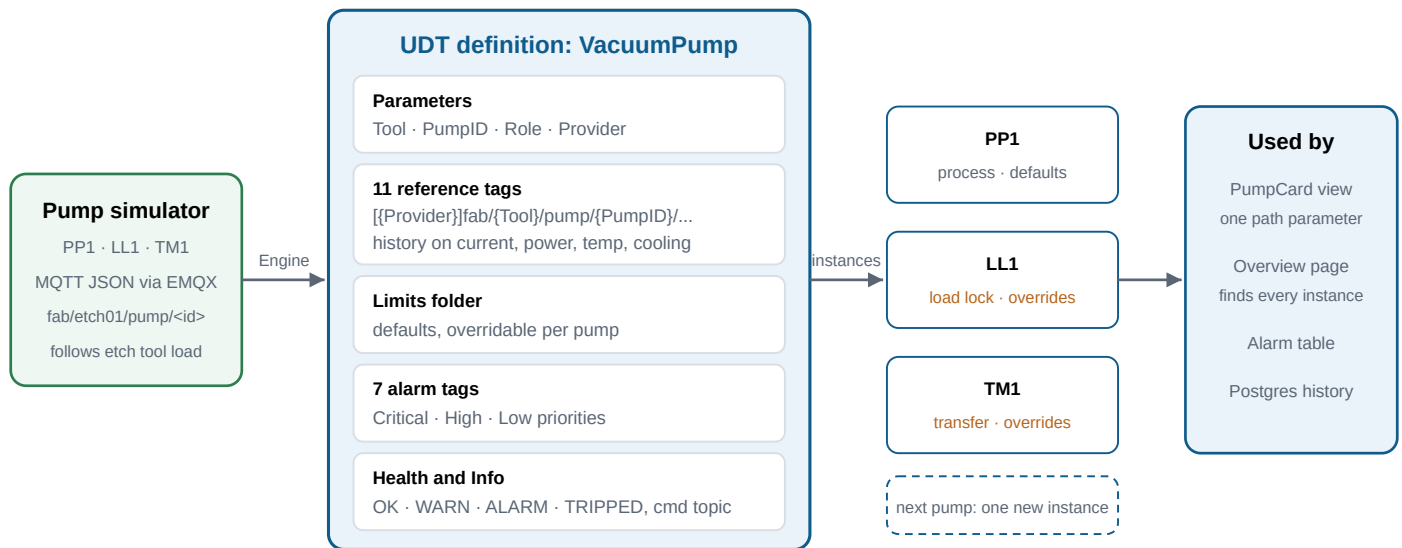
A VacuumPump UDT in Ignition 8.3: model once, deploy to any number of dry pumps

Christoph Viehoff · Automation and Controls Engineer · Portland, OR

Home lab build, September 2026

Fabs run hundreds of dry vacuum pumps, and every one needs the same data, limits, alarms, history and screen. This lab models the pump once as an Ignition User Defined Type (UDT). Each physical pump then becomes one instance with four parameters. Tags, alarms, history, health logic and the operator card all come from the definition. A change made in the definition reaches every pump at once.

How it fits together



Why a UDT

Consistency

- Every pump has the same tag names, units, formats and alarm set
- Screens, reports and analytics can rely on one structure
- No hand-built tags to drift apart over time

Change once, apply everywhere

- Edit a limit, alarm or expression in the definition and every instance updates at once
- Instances keep only their real differences as overrides, shown in bold

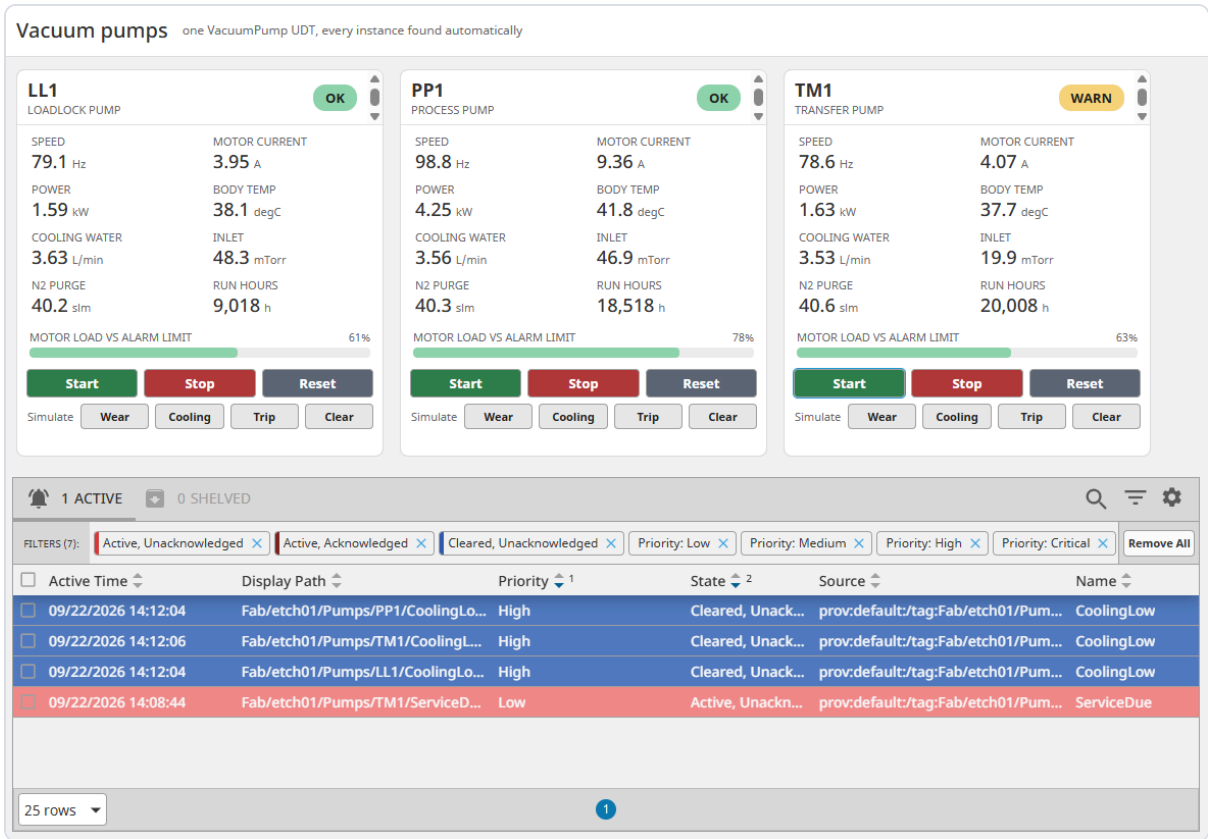
Fast to scale

- A new pump is one instance with four parameters
- The overview page browses for VacuumPump instances, so the new card appears with no screen work

Built-in intelligence

- Health state, load percentage and service due are computed in the tag layer, not in screens
- Alarms carry the pump's path, so operators see which pump at a glance

The result on screen



Ignition Perspective overview. Three cards from one PumpCard view, each bound to a different instance. TM1 shows WARN because it passed its 20,000 hour service interval, and the ServiceDue alarm is active. The blue rows are cleared start-up alarms, the timing case described on page 3.

Anatomy of the definition

Group	Members	Purpose
Parameters	Tool, PumpID, Role, Provider	The only per-pump inputs. They build every data path, the command topic and the labels.
Reference tags	State, Running, Speed, MotorCurrent, MotorPower, InletPressure, ExhaustPressure, BodyTemp, N2Purge, CoolingFlow, RunHours	Point at <code>[{Provider}]fab/{Tool}/pump/{PumpID}/<Metric>/value</code> . Units and formats set once. History to Postgres on the four trend signals.
Limits	CurrentWarn, CurrentAlarm, TempWarn, TempAlarm, MinCoolingFlow, ServiceHours	Memory tags with defaults. Smaller pumps override the current limits.
Alarm tags	Tripped, CurrentHighHigh, TempHighHigh, CoolingLow, CurrentHigh, TempHigh, ServiceDue	Boolean expressions against the limits, each with one alarm. Critical, High and Low priorities.
Derived	Health, CurrentLoadPct	One status word for screens and reports, and motor load as a percentage of the alarm limit.
Info	Tool, PumpID, Role, CmdTopic	Parameter values exposed as tags, so screens and scripts need only the instance path.

Demo scenarios

Scenario	What happens	What the UDT does
Bearing wear, PP1	Motor current climbs about 1.2 A per minute until the pump trips near 13.5 A	WARN at 10.5 A, High alarm at 12 A, Critical trip alarm. Load bar goes yellow, then red.
Loss of cooling, LL1	Cooling water drops to 0.9 L/min and body temperature climbs	CoolingLow High alarm at once, then TempHigh as the pump heats up
Service due, TM1	Run hours pass 20,000	Low priority ServiceDue alarm and WARN health, a hook for a maintenance work order
Process load, PP1	The etch tool starts PROCESSING	Current and power rise with gas load, showing the pump reacts to the tool it serves
Change for all	Lower TempWarn in the definition	All three pumps use the new limit at once, except where an instance has its own override

Adding a pump

- 1 In the Tags tab, right-click the Pumps folder and create a VacuumPump instance, for example PP2.
- 2 Set Tool and PumpID. Role and Provider keep their defaults.
- 3 Done. Tags, alarms, history and the overview card all appear. No screen changes.

Lessons from the build

- **Silent import loss.** Ignition 8.3 dropped the alarm list during import because of one unsupported field, with no error shown. I exported a hand-made alarm to get the gateway's exact format, then rebuilt the file from it.
- **Start-up race.** Running and CoolingFlow arrive as separate MQTT messages, so CoolingLow flickered on each start. The fix is a 5 second active delay on that alarm, set once in the definition for every pump.

How it scales to a fab

- One definition for each pump model, or a base type with child types for booster and dry stages
- Sub-fab monitoring for hundreds of pumps from one overview, filtered by tool or area
- Motor current and temperature history feeding predictive maintenance: rising current is an early sign of bearing wear
- ServiceDue and trip alarms routed to a CMMS as work orders
- The same pattern for chillers, abatement units, RF generators and gas boxes

Technology

SCADA	Ignition 8.3, UDTs, Perspective, alarming
Data path	Python sim, EMQX, MQTT Engine
History	PostgreSQL 17
Deployment	Docker Compose, JSON tag and view import